

Raspberry Pi Eurowings Flug-Checker

Handbuch - HA/FlightRadar24-Version und Legacy Daily/PDF-Version

Überblick

Die aktuelle Version liest Eurowings-Flüge aus einem ICS-Kalender. Home Assistant mit der FlightRadar24-Integration löst EW-Flugnummern in operative EWG-Callsigns auf. Der lokale ADS-B-Empfänger (dump1090) erkennt Überflüge. Die Landung des letzten Kalenderflugs wird in Phase 1 weiterhin über OpenSky verfolgt. Ereignis-Mails, Sonos-Audio und HomePilot-Licht bleiben unverändert Bestandteil der Logik.

Zwei Versionen im Repository

Datei	Einsatz
flug_checker.py	Aktuell: ICS + HA/FlightRadar24 + dump1090 + OpenSky
legacy/flug_checker_daily_pdf.py	Legacy: Daily-Mail + PDF/OCR statt Home Assistant

Aktuelle Architektur

Kalender -> EWxxxx + Route -> Home Assistant / FlightRadar24 -> EWG-Callsign -> callsigns.json (Cache) -> lokales dump1090 -> Überflug-Ereignis. Der letzte Flug wird ausschließlich aus dem Kalender bestimmt. Für dessen Landung sucht OpenSky anhand des EWG-Callsigns selbst die ICAO24 und verfolgt anschließend dieses Flugzeug.

Eine von HA/FR24 gelieferte ICAO24 wird nur akzeptiert, wenn sie aus genau sechs Hex-Zeichen besteht. Sie ist für Phase 1 nicht erforderlich, bleibt aber für einen späteren FR24-Landetracker vorbereitet.

Hardware

Raspberry Pi (z.B. Raspberry Pi 3), RTL-SDR-Stick mit 1090-MHz-Antenne; optional Sonos und Rademacher/HomePilot.

Installation

ADS-B / dump1090

```
sudo apt update
sudo apt install dump1090-mutability
# Daten: /run/dump1090-mutability/aircraft.json
```

Python-Umgebung

```
cd /home/pi
python3 -m venv flug_env
source flug_env/bin/activate
pip install requests pytz ics soco
```

Konfiguration

Geheimnisse liegen in /home/pi/.flugchecker_config. Diese Datei niemals in GitHub einchecken.

```
ICS_URL=
HA_URL=http://HOME_ASSISTANT_IP:8123
HA_TOKEN=
OPENSKY_USER=
OPENSKY_PASS=
MAIL_USER=
MAIL_PASSWORD=
MY_MAIL=
WIFE_MAIL=
HOME_LAT=
HOME_LON=
RADIUS_KM=25
INFO_RADIUS_KM=50
SONOS_IP=
PI_IP=
HTTP_PORT=8000
HOMEPILOT_URL=http://HOMEPILOT_IP/devices/DEVICE_ID
chmod 600 /home/pi/.flugchecker_config
```

Home Assistant / FlightRadar24

Benötigt werden `text.flightradar24_add_to_track` und `sensor.flightradar24_additional_tracked`. Neue EW-Flugnummern werden einmal pro Programmlauf an HA übergeben. Danach liest der Checker den HA-Sensor regelmäßig. `callsigns.json` wird nur geschrieben, wenn sich der Inhalt tatsächlich ändert.

Beispiel: EW765 -> EWG7RA

OpenSky - Phase 1

OpenSky wird nur für die Landung des letzten Kalenderflugs verwendet. Der Checker sucht den letzten EWG-Callsign, ermittelt darüber die ICAO24 und verfolgt danach dieses Flugzeug. Die bestehende Landelogik und die Ereignisaktionen wurden beim HA/FR24-Umbau nicht verändert.

Systemd

```
ExecStart=/home/pi/flug_env/bin/python /home/pi/flug_checker.py
sudo systemctl daemon-reload
sudo systemctl enable flugchecker
sudo systemctl start flugchecker
```

Wichtige Befehle

```
python3 -m py_compile /home/pi/flug_checker.py
systemctl status flugchecker.service --no-pager
journalctl -u flugchecker.service -f
tail -f /home/pi/flug_checker.log
/home/pi/flug_env/bin/python /home/pi/flug_checker.py --test
```

Legacy Daily/PDF/OCR

Die Legacy-Datei bleibt bewusst erhalten. Sie benötigt kein Home Assistant und gewinnt die Callsigns aus der Daily-Mail und dem PDF-Anhang. Dafür sind PDF-Text/OCR-Abhängigkeiten erforderlich.

```
sudo apt install tesseract-ocr poppler-utils
source /home/pi/flug_env/bin/activate
pip install pdf2image pytesseract pillow
```

Backup / GitHub

Das Repository enthält beide Varianten. `push.sh` übernimmt die aktuelle `/home/pi/flug_checker.py` und - falls vorhanden - `/home/pi/flug_checker_daily_pdf.py` in den Legacy-Ordner, führt einen Syntaxcheck aus und pusht anschließend alle Änderungen.

Phase 2

Nach stabilem Produktivbetrieb kann die OpenSky-Landeüberwachung durch FlightRadar24-Daten ersetzt werden. Bis dahin bleibt OpenSky bewusst aktiv.